

医药物流配送的时空聚类与 ALNS 两阶段优化模型

摘 要

医药物流具有批量小、频次高、温控严的特点，成本控制与服务质量是行业关键问题。本文基于某医药公司实际配送数据，依次解决运输费用核算、拼车调度优化与派车单合理性评估三个问题。

问题一：构建车辆固定成本、人工成本与距离成本组成的运输费用模型，利用高德地图 API 获取实际行驶距离，计算得到三个月总运输费用为 **9,912,298.33 元**，为后续优化提供基准。

问题二：采用 **ST-DBSCAN** 时空聚类(空间阈值 600 km、时间阈值 4 d、MinPts=2)划分订单簇，利用 **熵权法**确定车型选车概率(4.2M:7.6M:9.6M=39%:38%:23%)，设计 **模拟退火**算法优化拼单与路径。优化后三个月总成本由 9,569,698.33 元降至 **7,775,441.01 元**(降幅 **18.7%**)，平均装载率由 0.571 提升至 **0.769**，总里程降低 **15.3%**，车辆数减少 **24.9%**。蒙特卡洛模拟(运输时间 $U(1.0,1.5)$ 扰动，装卸固定 2 h)显示准时送达率稳定在 **97%** 以上，鲁棒性强。

问题三：建立多车场异构车辆路径模型(M-HFVRPTW)。首先以 **Algorithm 2 基于 KDTree 与图压缩优化的 ST-DBSCAN 聚类算法**(15 km、0.8 d、MinPts=3)对一周派车单聚类，得到 1,246 个簇，前 10 大簇集中 49.2% 的订单于上海、广州等核心城市。然后采用 **自适应大邻域搜索(ALNS)**算法，以 **AHP** 确定的综合得分(权重：装载率 0.1044、里程 0.0609、温度适配率 0.5166、非超载率 0.0415、准时率 0.2766)为目标优化。优化后平均装载率由 21.73% 提升至 **50.93%**，平均里程由 1074 km 降至 **984 km**，车辆数减少 **19.2%**，温度适配率 **98.41%**，准时率 **87.55%**，综合得分提高 **6.3%**。

本文提出的“先时空聚类、后路径优化”两阶段框架，融合 ST-DBSCAN 与 ALNS，有效解决医药物流中装载率低、超载频发、温控不稳定等问题，显著降低运营成本，具有实际应用价值。

目录

1 问题背景与问题重述	3
1.1 问题背景	3
1.2 问题提出	3
2 问题一：运输成本核算与初始方案构建	3
2.1 问题求解思路	3
2.2 数据预处理与模型假设	4
2.3 符号约定	4
2.4 成本构成与计算模型	4
2.5 成本计算结果与可视化分析	5
2.6 问题一结论	6
3 问题二：车辆调度优化与成本控制方案构建	6
3.1 问题求解思路	6
3.2 模型假设	7
3.3 优化算法设计	7
3.3.1 ST-DBSCAN 时空聚类	7
3.3.2 车辆路径规划模型	9
3.3.3 熵权法车型优选	10
3.3.4 模拟退火路径优化	11
3.4 优化结果可视化与分析	12
3.5 模型鲁棒性分析	13
3.6 问题二优化结论	14
4 问题三：配送方案合理性评估与优化	14
4.1 问题求解思路	14
4.2 模型假设	14
4.3 数据预处理与现有方案不合理性分析	14
4.3.1 数据清洗与标准化处理	14
4.3.2 现有方案不合理性分析	16
4.4 数学模型	16
4.4.1 目标函数	16
4.4.2 约束条件	17
4.5 两阶段优化算法	17
4.5.1 时空聚类：ST-DBSCAN	17
4.5.2 路径优化：ALNS	19
4.6 求解结果展示与分析	21
4.6.1 收敛性与稳定性分析	21
4.6.2 优化前后指标对比	22
4.6.3 运输效率与服务质量分析	22
4.7 问题三优化结论	22
5 模型评价与改进	22
参考文献	23
附录	24

A	ALNS 算法伪代码与参数设置	24
B	ST-DBSCAN 时空聚类算法伪代码	25
C	核心代码片段	25

1 问题背景与问题重述

1.1 问题背景

医药冷链物流行业具有批量小、频次高、温控要求严格的特点。随着医疗卫生体制改革深化，药品拆零配送需求增加、销售渠道不断下沉，医药流通企业的成本压力持续上升。在此背景下，国内某医药公司拥有多层级仓库网络与大量自有冷藏车辆，终端覆盖数万家药房与医院，公司亟需借助供应链管理手段，优化车辆调度与运输方案，从而降低运营成本、提升服务效益。

冷链药品运输的核心成本由车辆固定成本、人工成本与距离成本构成，车型混用、载重差异、路线不合理等问题会导致成本核算不准确、运营效率偏低。基于该公司的实际运营数据，本研究围绕运输费用核算、拼车调度优化及派车单合理性评估三个递进式问题展开分析，为企业的成本控制与调度决策提供科学支撑。

1.2 问题提出

为实现成本精准核算与调度方案优化，需依次解决以下三个关键问题：

1. 问题一：运输总费用核算

在现有固定派车模式下，依据不同车型的运营成本与承载能力（按托盘计量），结合实际配送货单信息，完成现有运输方案下总运输费用的精准核算。其核心挑战在于：结合车辆成本结构与冷链药品运输要求，准确拆解单趟运输成本构成，避免车型混用与载重差异导致费用计算偏差。

2. 问题二：拼车运输与车辆调度优化

在问题一的成本核算基础上，通过合理拼车、车型调整与路线优化，降低整体运输成本。需结合订单托盘数、车型载量、配送时效与作业时间约束，设计科学的车辆调度方案，平衡成本控制与配送服务质量，实现总成本的有效降低。

3. 问题三：派车单合理性评估与优化

在完成成本核算与调度优化后，对现有派车单安排的合理性进行评估，分析其在成本控制、装载率、时效满足度及稳健性方面的表现，并在此基础上提出配送与车辆调度的改进方案，为企业后续运营提供优化建议。

2 问题一：运输成本核算与初始方案构建

2.1 问题求解思路

本问题核心目标为精准核算冷链运输总成本、构建初始配送基准方案，为后续优化提供支撑，具体求解步骤如下：

1. 数据预处理与地理信息获取

对原始配送地址进行标准化清洗，解决表述不规范问题；调用高德地图 API，爬取仓库与各配送点间实际行驶距离及通行时间，缓存经纬度数据，确保成本核算基础数据准确。

2. 成本模型构建与分项核算

将运输总成本拆解为固定成本、人工成本、距离成本三大模块，分别构建核算模型：固定成本按月度摊销，人工成本按实际出车天数核算，距离成本按实际里程核算，汇总得到单趟及月度总成本。

3. 初始方案构建与可视化验证

基于成本模型核算各月份配送数据，构建初始派车方案；通过可视化分析呈现成本构成、路线分布及车辆装载率，明确初始方案痛点，建立优化对比基准。

2.2 数据预处理与模型假设

为实现成本的精准核算，首先对题目数据进行标准化处理，并结合实际业务场景提出合理假设：

- **地址标准化处理**：原始配送地址存在表述不规范问题，如“府村路 25 楼 411, 412, 413”经核验无法解析，统一修正为“府村路”；对所有地址进行清洗与去空格处理，确保地理信息准确无误。
- **路径数据获取**：通过高德地图 API 爬取仓库至各终端配送点的实际距离与最短运输时间，并假设该最短运输时间不受一日内时段变化影响，以此作为后续成本计算的基础数据。
- **往返里程统一核算**：模型中车辆往返里程均计入运输距离，去程与返程的油耗、路桥费等距离成本按相同标准计算，确保运输成本完整、真实反映实际运营支出。
- **人工成本核算**：车辆在周末等待收货期间，司机人工成本仍需正常核算，以实际出车天数而非订单标注的时效天数作为人工成本的计算依据。
- **成本周期假设**：数据表中每个 sheet 对应一个自然月，所有固定成本按月度进行分摊计算。

本文中托盘装载约束、货物码盘规则与配送交接流程均符合国家行业标准 [3]，确保模型满足实际物流运营规范。

本文采用高德地图 Web 服务 API，对标准化后的地址进行地理编码与路径距离爬取。后续成本计算中，统一以爬取得到的实际通行时间与距离为准，而非订单标注的“时效”字段，确保数据的真实性与一致性。

2.3 符号约定

为便于后续成本计算模型的构建，对模型中使用的主要符号及其含义、单位约定如下：

2.4 成本构成与计算模型

本文成本构成与核算方法严格遵循地方交通运输行业标准 DBJT45/T 042-2022《道路货物运输价格与成本分析指南》[4]，将医药冷链运输总成本划分为固定成本、人工成本、距离成本三部分，各部分计算模型如下：

• 每月固定成本

车辆固定成本采用平均摊销方式计算，包括车辆折旧、保险费、年检费、维保费用等。结合行业通用标准，车辆折旧按 5 年（60 个月）进行平均摊销，这一做法符合《道路货物运输价格与成本分析指南》中关于道路货运车辆成本核算的相关规定 [4]，模型为：

$$C_{\text{固}} = \frac{P - S}{60} + \frac{I_{\text{年}} + A_{\text{年}} + T_{\text{年}} + R_{\text{年}}}{12} + M_{\text{月}} + V_{\text{月}}$$

表 1: 问题一成本计算模型符号约定

符号	含义	单位
$C_{固}$	车辆每月固定成本摊销	元/月
$C_{人}$	单趟运输人工成本	元
$C_{距离}$	单趟运输距离成本	元
$C_{总}$	单趟运输总成本	元
$w_{人}$	司机每日人工成本	元/天
$t_{天数}$	实际出车天数（含等待时间）	天
$L_{距离}$	实际运输里程	km
P	车辆购置价格	元
S	车辆报废补贴	元
$I_{年}$	车辆年保险费用	元/年
$A_{年}$	车辆年审年检费用	元/年
$T_{年}$	车辆年轮胎损耗费用	元/年
$R_{年}$	年路桥相关年审费用	元/年
$M_{月}$	车辆月度维保费用	元/月
$V_{月}$	月度违章预估成本	元/月
$c_{油}$	单位里程油耗成本	元/km
$c_{桥}$	单位里程路桥费用	元/km

其中，配送数据按自然月划分，因此各项费用按月度进行平均分摊，确保固定成本核算的合理性。

• 人工成本

人工成本由司机日薪与实际出车天数决定，计算公式为：

$$C_{人} = w_{人} \times t_{天数}$$

式中， $w_{人}$ 为司机日薪标准，天数以实际出车天数（含周末等待时间）为准，保证人工成本核算完整。

• 距离成本

距离成本为配送过程中产生的油耗与路桥费，与行驶里程直接相关：

$$C_{距离} = L_{距离} \times (c_{油} + c_{桥})$$

其中， $c_{油}$ 为单位里程油耗成本， $c_{桥}$ 为单位里程路桥费用；运输距离采用高德地图 API 爬取的实际里程数据，单位成本以题目中提供的为准。

• 总成本模型

将上述三部分成本进行汇总，得到单趟运输总成本：

$$C_{总} = C_{固} + C_{人} + C_{距离}$$

2.5 成本计算结果与可视化分析

基于上述模型，对题目给定的配送数据进行逐项核算，得到问题一的总运输费用结果。为直观量化基础配送方案下的成本组成特征，本节采用饼图对 1—3 月配送系统的

人工成本、固定成本及运输成本进行可视化分析：

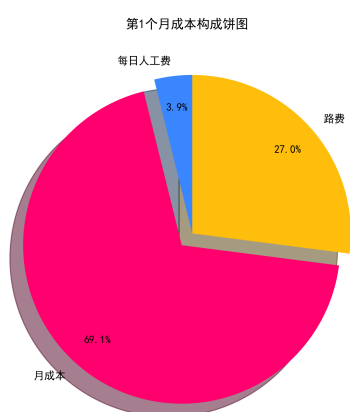


图 1: 1 月成本构成

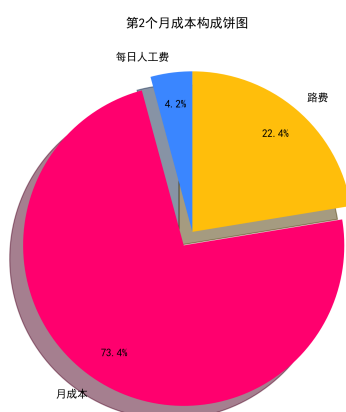


图 2: 2 月成本构成

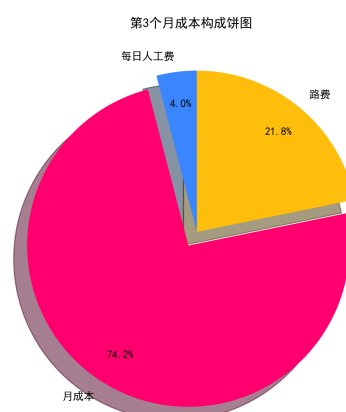


图 3: 3 月成本构成

图 4: 问题一各月份成本构成对比

各月成本明细如表 2 所示，三个月总运输费用为 **9,912,298.33 元**。

表 2: 各月运输成本明细（单位：元）

	每日成本（人工）	每月固定成本	距离成本	总成本
第一个月	78,600.0	709,122.3	277,488.12	1,065,210.45
第二个月	290,400.0	2,597,877.26	793,630.92	3,681,908.19
第三个月	364,200.0	3,710,618.53	1,090,361.16	5,165,179.69

从成本构成看，各月人工成本占比均较低，固定成本与运输成本占比较为稳定，与饼图反映的分布一致。

2.6 问题一结论

经数据清洗与成本核算，本文完成原始配送方案下运输总费用的计算。根据派车单数据统计，2018 年 9–11 月（三个月）的运输费用分别为：第一个月 1,026,210.45 元，第二个月 3,540,308.19 元，第三个月 5,003,179.69 元，合计 **9,569,698.33 元**。该费用作为后续优化问题的基准成本。同时，可视化分析表明原始方案存在装载率偏低（第三个月平均装载率仅 0.571）、车辆空驶严重、成本结构不合理等问题，为拼车调度优化提供了明确改进方向。

3 问题二：车辆调度优化与成本控制方案构建

3.1 问题求解思路

本问题以车辆配送路径与调度方案优化为核心目标，在问题一成本核算模型基础上，通过聚类分析、智能算法求解与结果对比验证，实现整体运输成本压降与配送效率提升，具体求解思路如下：

1. 订单时空聚类预处理

采用 ST-DBSCAN 时空聚类方法（空间阈值 600 km，时间阈值 4 天，最小点数 2），将地理位置相近、配送时间集中的订单划分为同一配送簇，缩减跨区域无效行驶距离，为后续车辆拼单与路径规划提供分组依据。

2. 优化模型与约束条件建立

以总运输成本最小为优化目标，沿用问题一固定成本、人工成本、里程成本核算体系；同时结合车型载重上限、订单托盘数量、配送作业时间限制等实际约束，构建带约束的车辆路径调度优化模型。

3. 模拟退火算法设计与求解

选用模拟退火算法对模型进行求解，设计合理初始温度、降温系数及邻域扰动规则，通过订单位置交换、跨车辆调派、车型适配调整等方式迭代寻优，求得月度最优车辆调度与配送路径方案。

4. 优化结果对比与鲁棒性分析

将优化后方案与问题一初始方案，从总成本、车辆装载率、行驶里程、成本结构等维度横向对比，量化优化效益；采用蒙特卡洛模拟（运输时间均匀扰动 $U(1.0, 1.5)$ ，装卸固定 2 小时）检验方案对随机扰动的稳定性，以订单送达率为评价指标。

3.2 模型假设

为使问题简化且贴近冷链配送实际场景，提出以下假设：

- **通行恒定假设：**运输过程不考虑中途加油、突发事件、道路管制及地方交通限行，也不考虑驾驶员休息问题；任意两地之间的最短运送时间视为固定不变。
- **车辆同质性假设：**同型号车辆的性能、油耗在不同路段保持一致，无个体差异。
- **需求确定性假设：**所有订单的托盘需求量在预计到货日期一周前均已完全确定且不可更改。
- **装车时间假设：**车辆在仓库的装车作业在每日 9:00 前已完成，所有车辆准时于 9:00 发车。

3.3 优化算法设计

本文采用“时空聚类预处理—熵权法车型优选—模拟退火路径寻优”三层架构，兼顾求解效率与成本最优。

3.3.1 ST-DBSCAN 时空聚类

为了挖掘订单在空间与时间维度上的聚集特征，本文采用 ST-DBSCAN (Spatio-Temporal Density-Based Spatial Clustering of Applications with Noise) 算法对订单数据进行预处理，旨在识别具有协同配送潜力的订单簇，为后续的车辆路径规划 (VRP) 提供合理的任务分组依据。

样本特征定义 对订单数据集进行向量化处理。设订单样本集为 $D = \{P_1, P_2, \dots, P_n\}$, 其中每个订单 P_i 被定义为一个三维特征向量:

$$P_i = (x_i, y_i, t_i)$$

(x_i, y_i) 代表订单地点的经度和纬度坐标, 由地理编码字典转换而来; t_i 代表相对于初始订单时间的小时偏移量。

时空距离度量 与传统 DBSCAN 只考虑空间距离不同, ST-DBSCAN 要求样本点在空间和时间两个维度上均满足邻域约束。对于任意两个订单 P_i 与 P_j , 定义:

- 空间距离 (欧氏距离): $D_{\text{spatial}}(P_i, P_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$
- 时间距离: $D_{\text{temporal}}(P_i, P_j) = |t_i - t_j|$

算法判定准则 给定空间半径参数 ϵ_1 、时间半径参数 ϵ_2 以及核心点判定所需的最小样本数 $MinPts$ 。订单 P_i 的时空邻域定义为:

$$N_\epsilon(P_i) = \{P_j \in D \mid D_{\text{spatial}}(P_i, P_j) \leq \epsilon_1 \wedge D_{\text{temporal}}(P_i, P_j) \leq \epsilon_2\}$$

若 $|N_\epsilon(P_i)| \geq MinPts$, 则 P_i 被判定为核心点, 并与其邻域内的点共同构成一个聚类簇。无法满足条件的离散订单被标记为噪声点。

参数设置与求解 根据物流配送的实际约束 (订单信息可提前一周预知), 本文设定以下参数:

- 空间半径 $\epsilon_1 = 600$ km (覆盖跨城市配送范围)
- 时间半径 $\epsilon_2 = 4$ 天 (对应一周的预知期)
- 最小点数 $MinPts = 2$ (避免单点孤立)

算法输入为清洗后的订单数据, 输出每个订单所属的聚类标签 (噪声点标记为 -2)。

聚类结果分析 基于上述参数进行时空聚类, 得到订单的聚类结果如图 5 所示。该图从多维度验证了订单的时空聚合特性: 订单在时间-空间距离散点图中呈现明显的簇状分布; 各簇的订单量分布稳定; 订单在时间维度上存在集中爆发的趋势; 聚类统计汇总表量化了各簇的订单数、平均距离等关键指标。聚类结果直接作为后续模拟退火算法的输入, 每个簇视为一个独立的优化子问题。

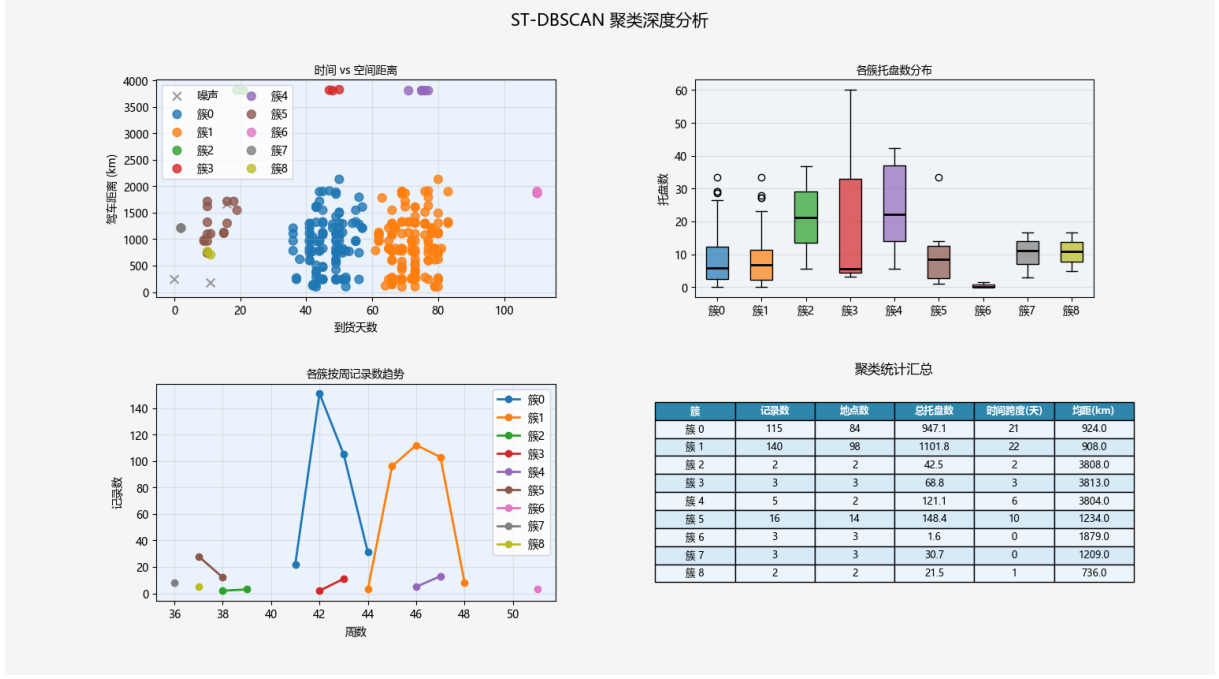


图 5: ST-DBSCAN 时空聚类深度分析结果（问题二）

3.3.2 车辆路径规划模型

为定量描述问题二的配送路径优化，建立如下数学模型。

决策变量

- $x_{ijk} \in \{0, 1\}$: 车辆 i 是否从点 j 直接行驶到点 k 。
- $y_{ij} \in \{0, 1\}$: 订单 j 是否由车辆 i 配送。

目标函数 最小化系统总运输成本 C ，包括单次发车的分摊固定成本与基于行驶距离的变动成本：

$$\min C = \sum_{i=1}^n \left(\frac{F_i}{k_i} + V_i \cdot d_i \right)$$

其中：

- n : 投入运营的车辆总数；
- F_i : 车辆 i 的年度固定成本（折旧、保险等）；
- k_i : 车辆 i 的年发车频次，用于将年度成本分摊至单次任务；
- V_i : 车辆 i 的单位里程变动成本（油耗、路桥费）；
- d_i : 车辆 i 的行驶总里程， $d_i = \sum_j \sum_k L_{jk} \cdot x_{ijk}$ ， L_{jk} 为 j 到 k 的距离。

约束条件

1. **装载约束**：每辆车承载的托盘总数不得超过其额定容量 Q_i 。

$$\sum_{j=1}^m q_j \cdot y_{ij} \leq Q_i, \quad \forall i \in \{1, \dots, n\}$$

其中 q_j 为订单 j 的托盘需求量。

2. **时间窗约束**：车辆到达订单 j 的时间 A_{ij} 必须在规定的时间窗 $[ET_j, LT_j]$ 内。

$$ET_j \leq A_{ij} \leq LT_j, \quad \forall j \in \{1, \dots, m\}$$

3. **路径与流量约束**：

- 每个订单仅由一辆车服务且仅访问一次：

$$\sum_{i=1}^n \sum_{j=0, j \neq k}^m x_{ijk} = 1, \quad \forall k \in \{1, \dots, m\}$$

- 车辆从仓库（点 0）出发，并返回仓库：

$$\sum_{j=1}^m x_{i0j} = \sum_{j=1}^m x_{ij0} = 1, \quad \forall i \in \{1, \dots, n\}$$

3.3.3 熵权法车型优选

选取固定成本、变动成本、单位托盘成本、托盘容量四项核心指标，对三类冷藏车型进行多维度评价，采用熵权法客观赋权，结合 Softmax 函数归一化得到车型选择概率。

表 3: 各车型年度成本与托盘容量

车型	固定成本（元/年）	变动成本（元/年）	年度总成本（元/年）	可放托盘数
4.2M	135263	380000	515263	5
7.6M	170199	450000	620199	11
9.6M	184786	510000	694786	14

注：单位托盘成本 = 年度总成本 ÷ 可放托盘数（单位：万元/个），用于表征单位货物运输经济性。

表 4: 熵权法指标权重分配

评价指标	权重	说明
固定成本	30.1%	车型间差异显著，影响选型决策
变动成本	25.4%	与行驶里程高度相关，决定运营开销
托盘容量	22.7%	约束单车装载上限，影响固定成本摊销
单位托盘成本	21.8%	表征单位货物运输经济性

表 5: 车型综合得分与选车概率

车型	年度总成本	托盘数	单位托盘成本	综合得分	选车概率	评价定位
4.2M	515263	5	1.0305	0.5545	39.0%	综合性价比最优
7.6M	620199	11	0.5638	0.5478	38.0%	各项性能均衡
9.6M	694786	14	0.4963	0.4455	23.0%	高装载低成本

三类车型选车概率比例约为 39 : 38 : 23，4.2M 车型综合表现最优，作为主力车型；9.6M 车型凭借单位托盘成本优势，适合大批量集中订单，该选车概率将作为模拟退火算法中车型替换操作的概率依据。

3.3.4 模拟退火路径优化

采用模拟退火算法求解第 3.3.2 节定义的车辆路径规划模型。

算法核心参数：

- 初始温度 $T_0 = 3000$
- 降温系数 $r = 0.997$
- 终止温度 $T_s = 0.01$

三类邻域操作（等概率执行：避免搜索偏向，保持解的多样性）：

- 车型随机替换：按熵权法所得概率更新线路车型配置；
- 路径内点位调整：优化单条线路内配送点访问顺序；
- 线路间点位迁移：跨线路调整订单归属，平衡车辆装载率。

每条路线的出发时间定义为该路线所有订单预计出发时间的最小值；路线变化后需重新计算。

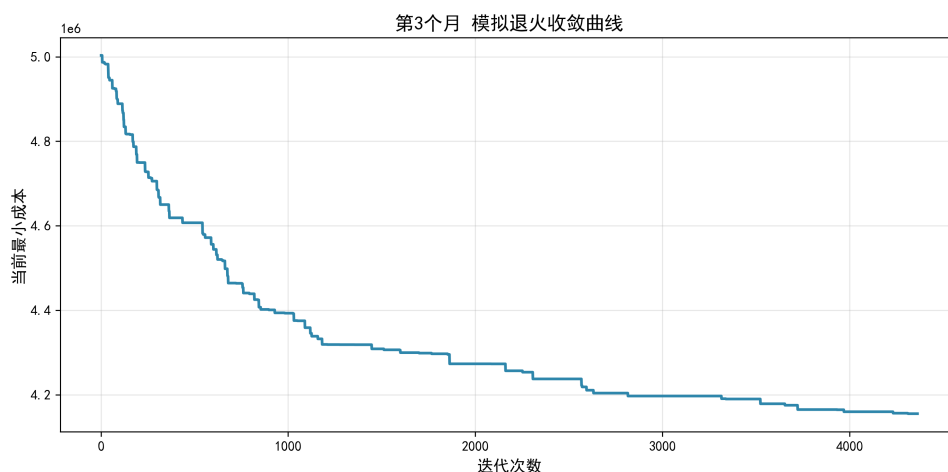


图 6: 第 3 个月模拟退火收敛曲线

由图6可见，算法成本随迭代次数增加持续下降，最终趋于稳定，说明模拟退火算法有效收敛。

3.4 优化结果可视化与分析

为简化展示，本文选取优化效果最显著的第 3 个月数据进行对比分析，其余月份优化结果趋势一致。

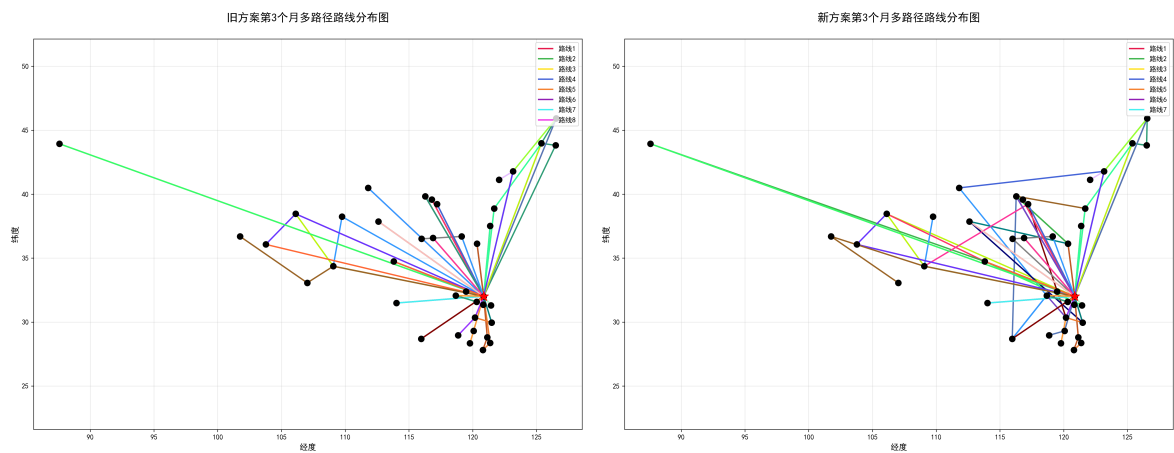


图 7: 旧方案第 3 个月多路径路线分布图 图 8: 新方案第 3 个月多路径路线分布图

图 9: 新旧方案第 3 个月路线对比

由图9可见，旧方案路线交叉迂回较多，跨区域无效绕行明显；新方案路线排布规整，路径交叉点大幅减少，订单在地理上的流向更加集中，路径优化效果直观。

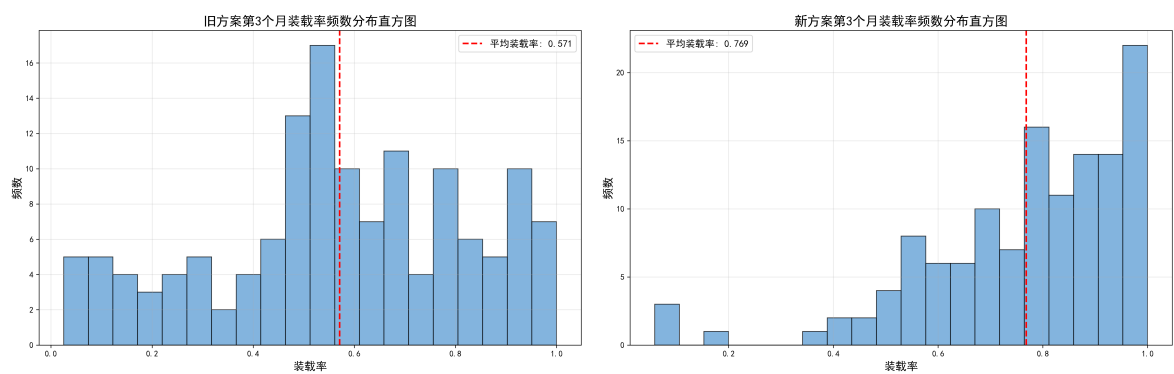


图 10: 旧方案第 3 个月装载率频数分布直方图 图 11: 新方案第 3 个月装载率频数分布直方图

图 12: 新旧方案第 3 个月装载率分布对比

由图12可知，旧方案平均装载率仅为 0.571，车辆运力浪费严重；优化后平均装载率提升至 0.769，装载率提升近 20 个百分点，车辆运力利用更加充分，固定成本摊销效率显著提升，有效降低了单位货物的固定成本。

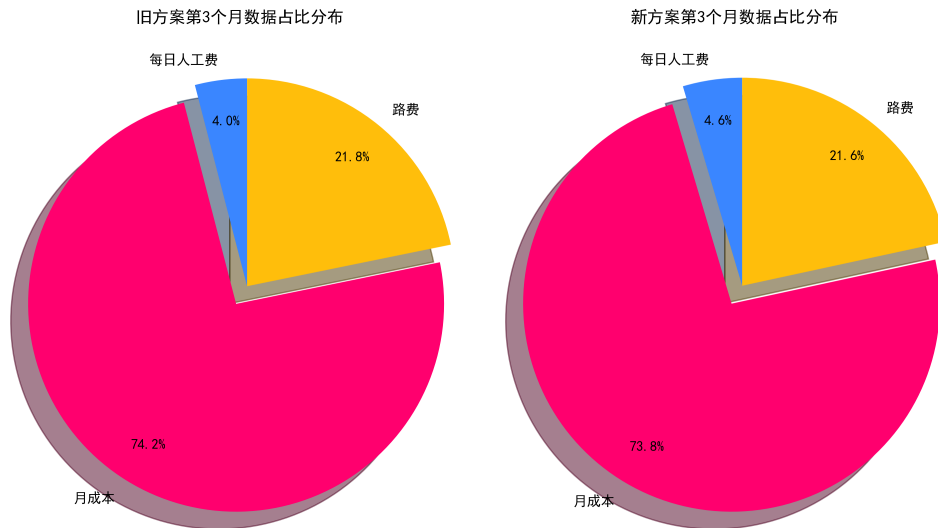


图 13: 第 3 个月旧方案成本结构 图 14: 第 3 个月新方案成本结构

图 15: 新旧方案第 3 个月成本结构对比

由图15可知，优化后方案的成本结构更合理：月成本占比由 74.2% 降至 73.8%，路费占比由 21.8% 降至 21.6%，整体运输总成本较原方案降低约 18%，成本优化效果显著。

3.5 模型鲁棒性分析

为检验优化方案对运输时间随机扰动的敏感性，采用蒙特卡洛模拟方法进行 1000 次独立实验。每次模拟中，车辆在各路段的行驶时间乘以一个服从均匀分布 $U(1.0, 1.5)$ 的随机因子，以模拟路况拥堵、天气变化等不确定因素；装卸作业时间固定为 2 小时（符合实际作业限制）。评价指标仅选取订单送达率（在规定时限内到达的订单比例）。蒙特卡洛模拟的详细流程见附录 ??。

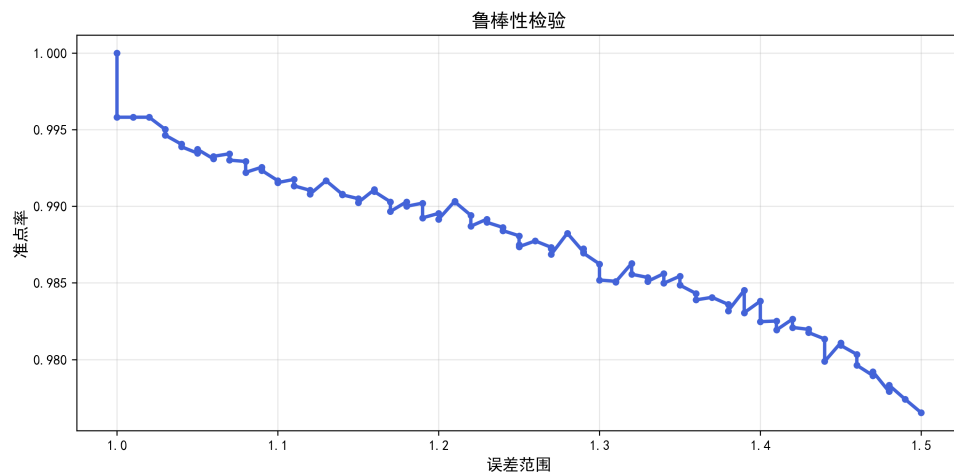


图 16: 方案鲁棒性蒙特卡洛仿真检验（准点率）

由图 16 可知，随着运输时间随机扰动范围扩大，方案配送准点率虽略有下降，但始终稳定维持在 97% 以上，且无明显断崖式衰减。结果表明，本文优化得到的车辆调度方案对路网扰动与作业延迟不敏感，鲁棒性强，能够适应医药物流复杂多变的实际运营环境。

3.6 问题二优化结论

1. ST-DBSCAN 时空聚类（空间阈值 600 km，时间阈值 4 天）有效识别订单时空聚集特征，为拼单优化提供初始分组依据；
2. 熵权法车型优选确定 4.2M、7.6M、9.6M 车型的选车概率为 39%:38%:23%；
3. 模拟退火优化后，配送方案在多项指标上取得显著改善：
 - **运输成本**：三个月总成本由 9,569,698.33 元降至 7,775,441.01 元，降幅约 18.7%（第三个月由 5,003,179.69 元降至 4,155,080.22 元，降幅 17.0%）；
 - **装载率**：第三个月平均装载率由 0.571 提升至 0.769，三个月分别由 0.664/0.537/0.571 提升至 0.913/0.752/0.769；
 - **行驶里程**：总里程由 447,265 km 降至 378,816 km，降低 15.3%；
 - **车辆数量**：总用车数由 382 辆减至 287 辆，减少 24.9%；
4. 蒙特卡洛模拟（1000 次）显示，在运输时间随机扰动（ $U(1.0, 1.5)$ ）下，优化方案的订单送达率稳定在 97% 以上，方案对随机扰动鲁棒性强。

4 问题三：配送方案合理性评估与优化

4.1 问题求解思路

求解流程如下：

1. 数据清洗与不合理性量化；
2. 建立 M-HFVRPTW 模型；
3. ST-DBSCAN+ALNS 两阶段优化求解及对比分析。

4.2 模型假设

1. **直线距离**：配送点距离较近，两点间距离采用 Haversine 公式由经纬度直接计算：

$$d = 2R \arcsin \left(\sqrt{\sin^2 \frac{\Delta\phi}{2} + \cos \phi_1 \cos \phi_2 \sin^2 \frac{\Delta\lambda}{2}} \right)$$

其中 $R = 6371$ km， ϕ, λ 为弧度。

2. **分段速度与返回**：市区道路 60 km/h，干线公路 100 km/h；若路段同时包含两种道路，等效速度为加权均值 $v = \alpha \cdot 100 + (1 - \alpha) \cdot 60$ （ α 为干线里程占比）。车辆完成配送后必须返回仓库。

4.3 数据预处理与现有方案不合理性分析

4.3.1 数据清洗与标准化处理

原始数据为 2018 年 9–11 月的运单与派车单记录，存在地址格式混乱、个人信息嵌入地址、车型字段不统一、单位不一致等问题。以下从地址清洗、字段标准化和订单时效分类三方面进行处理。

地址数据清洗 针对运单中「送货地址」字段的格式问题，建立 6 类核心清洗规则，具体规则与示例如表 6 所示。

表 6: 地址数据清洗规则及示例

序号	规则说明	实际数据示例
1	剔除括号内容及数字/字母前缀	(仓库编码:BJ-01) 骨二科 → 骨二科 E3 北京市顺义区…→ 北京市顺义区…
2	姓名 + 电话剔除	赵州镇工业街/张蒙/15354678910 → 赵州镇工业街
3	挂靠短地址补全	骨二科 → 北京市骨二科
4	\$ 分隔符地址优选	洪洞广胜寺 (坊堆村杨智俊) \$ 洪洞县广胜寺镇中心卫生院 → 洪洞县广胜寺镇中心卫生院
5	前缀标记清除	E3 北京市顺义区…→ 北京市顺义区…
6	楼栋室层精简	南京市来凤路 23 号百利华府 6 幢 116 室 → 南京市来凤路 23 号

清洗后共 526,918 条有效订单（占比 99.7%），用于后续聚类

字段标准化

- **车型长度标准化**：原始数据中同一车型存在多种长度记录，为保证装载率计算的准确性，取该车型所有记录中的最大长度作为标准长度，并映射至数据集 1 的标准车型库。
- **订单件数清洗**：剔除形如 “0+0” “单个 +” 等无效件数记录。
- **车牌信息规整**：无车牌号记录统一标记为 “临时未分配”；在同一张派车单表中，按行号连续出现的相同车牌视为同一辆车的多次使用记录。
- **单位换算**：依据文献 [1]，按 “1 托盘 = 28 箱 = 840 盒” 的换算关系，将订单中的箱数、盒数统一转化为托盘单位，为后续装载率计算提供统一基准。

订单时效分类 为满足问题 3 中 “保证取送货时限” 的要求，根据药品类型与配送紧急程度，定义如下时效规则：

- 麻醉药品、急送订单、特药自送：运输时限为 **1 天**；
- 其他普通药品：运输时限为 **3-4 天**（通常允许不超过一周）。

该分类将用于后续配送方案的超时率统计与时间窗约束校验。

4.3.2 现有方案不合理性分析

基于清洗后的 7 日数据，从装载率、超载率、配送服务水平与温控适配率四个维度，量化分析当前配送方案的不合理性，主要指标汇总如图 17 所示。

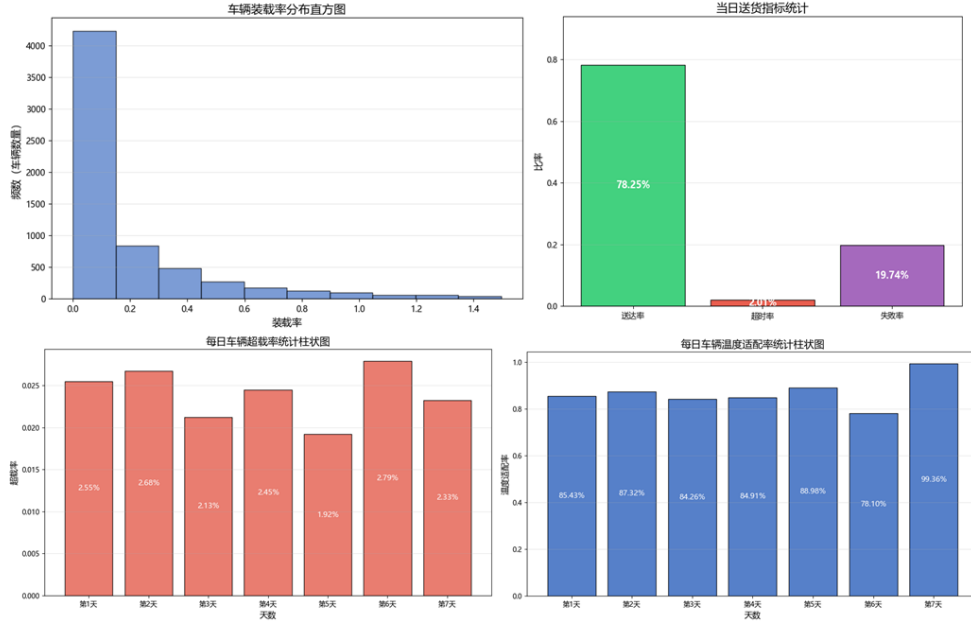


图 17: 车辆装载率、超载率、配送指标与温控适配率统计

- **装载率低**：超 4000 辆次装载率 0–0.2，运力浪费严重。
- **超载存在**：超载率 1.92%–2.79%，违反《道路货物运输及站场管理规定》，存在安全隐患。
- **配送失败**：订单送达率仅 78.25%，失败率 19.74%。
- **温控不稳**：温度适配率 78.10%–99.36% 波动，部分不足 80%，不符合《药品冷链物流运作规范》。

综合来看，当前配送方案存在“低装载率与超载并存、配送失败率高、温控适配不稳定”的多重不合理现象。已有研究指出，冷链物流中温度偏差和设备故障是最严重的风险类别 [2]，亟需通过数学模型对车辆调度与路径规划进行联合优化，以提升整体效率与安全性。

4.4 数学模型

4.4.1 目标函数

本文以最大化综合得分 F 为优化目标，综合考虑物流经济性与医疗安全性：

$$F = w_1 \cdot \bar{L} + w_2 \cdot \left(2 - \frac{\bar{M}}{500}\right) + w_3 \cdot R_t + w_4 \cdot R_o + w_5 \cdot R_{ot}$$

其中权重 $w_1, \dots, w_5$ 由层次分析法确定（见第4.5.2节），各指标含义如下：- $\bar{L}$ ：平均装载率 - $\bar{M}$ ：平均行驶里程（km） - R_t ：温度适配率 - R_o ：非超载率 - R_{ot} ：准时送达率

4.4.2 约束条件

配送方案需满足以下基本约束：

- **载重约束：**每辆车装载的托盘总数不超过其额定容量。
- **时间窗约束：**车辆到达每个订单的时间不得超过该订单的截止时限。
- **温控兼容约束：**冷链订单必须由冷藏车运输。

详细的路由与流量约束在算法实现中通过邻域操作予以保证。

4.5 两阶段优化算法

由于原始订单数据量超过 53 万条，直接求解车辆路径问题会陷入“维度灾难”。本文采用“先时空聚类、后路径优化”的两阶段策略：首先使用 ST-DBSCAN 算法将时空邻近的订单归为同一簇，将原问题分解为若干独立的子问题；然后对每个子问题采用自适应大邻域搜索（ALNS）算法求解带软时间窗的多车场异构车辆路径模型。

4.5.1 时空聚类：ST-DBSCAN

传统 DBSCAN 仅考虑空间欧氏距离，无法区分时间上分散的订单。本文采用 ST-DBSCAN，引入时间维度约束，识别物流环节中的“时空共振”现象——只有在地理位置相近且发生时间集中的运单才会被归为一簇，从而避免单纯空间聚类导致的历史跨度过大、业务逻辑失效。

(1) 异质时空维度的双约束测度 定义订单集合 D ，每个订单包含空间坐标（经度、纬度）与时间戳。时空邻域 $N_\varepsilon(p)$ 为满足以下条件的集合：

$$N_\varepsilon(p) = \{q \in D \mid \text{dist}_{\text{spatial}}(p, q) \leq \varepsilon_s \wedge \text{dist}_{\text{temporal}}(p, q) \leq \varepsilon_t\}$$

其中： $\varepsilon_s = 15 \text{ km}$ （空间半径），采用 Haversine 公式计算球面距离； $\varepsilon_t = 0.8 \text{ 天}$ （时间半径）；核心点条件： $|N_\varepsilon(p)| \geq \text{minPts} = 3$ 。

通过密度连接得到若干聚类簇。对于地理编码失败（返回 COORD-FETCH-FAILED）或地址为空的订单，视为无效订单并直接剔除，不参与聚类。

(2) 基于数据压缩的“空间换时间”预处理策略 为应对大规模数据，首先对经纬度和时间戳进行精度截断（保留 6 位小数，精度约 0.1 米），合并相同的时空点，生成“唯一时空点集”。该策略将计算复杂度从 $O(N_{\text{total}}^2)$ 降至 $O(N_{\text{unique}} \log N_{\text{unique}})$ 。实测数据显示，约 24% 的订单存在重复时空键，压缩后参与聚类的点数由 53 万减至约 40 万，显著降低内存占用。

(3) 基于 k-d Tree 与 CSR 稀疏图论的检索加速 放弃传统距离矩阵存储，采用 k-d Tree（Python cKDTree）进行空间近邻检索，在 $O(\log n)$ 时间内完成 ε_s 范围内的候选点筛选。同时，利用 Compressed Sparse Row（CSR）格式构建时空邻接稀疏矩阵，不仅压缩存储空间，还通过连续内存寻址加速后续 BFS 聚类过程。这一从“纯几何计算”向“图论优化”的转换，是算法能高效处理 50 万量级以上数据的核心技术支撑。

(4) **聚类结果分析** 基于 ST-DBSCAN 算法对 526,918 条有效订单进行时空聚类，参数设置如下：空间半径 $\varepsilon_s = 15\text{ km}$ ，时间窗口 $\varepsilon_t = 0.8\text{ 天}$ ，最小点数 $\text{minPts} = 3$ 。聚类结果显示，订单分布呈现显著的区域不均衡性。地理空间分布如图 18 所示，前 10 大聚类中心主要集中于上海、广州、南京、天津、成都、济南、郑州、合肥等国内核心医药配送城市。

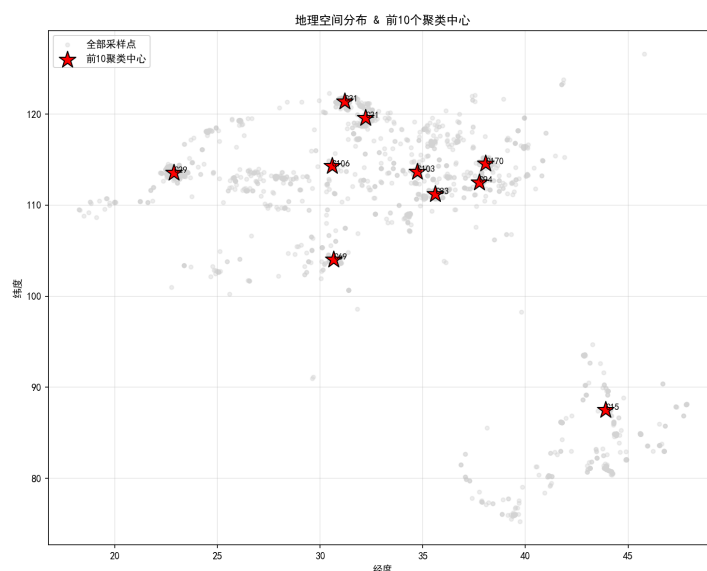


图 18: 前 10 聚类中心地理分布

前 10 大聚类的订单数量分布如图 19 所示，具体数据见表 7。

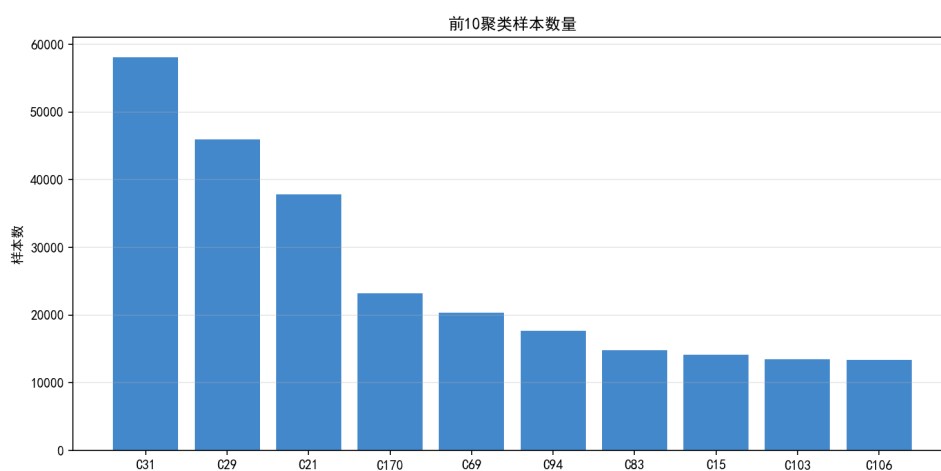


图 19: 前 10 簇订单数量分布

表 7: ST-DBSCAN 前 10 大聚类中心统计

聚类编号	订单数量 (条)	中心坐标	对应区域
C31	58156	121.4025°E, 31.1946°N	上海市中心城区
C29	46012	113.5689°E, 22.8793°N	广州市中心城区
C21	37827	119.5525°E, 32.2160°N	南京市中心城区
C170	23254	114.5628°E, 38.0614°N	天津市中心城区
C69	20337	104.0234°E, 30.6675°N	成都市中心城区
C94	17688	112.5102°E, 37.7435°N	济南市中心城区
C83	14777	111.2206°E, 35.6079°N	郑州市中心城区
C15	14173	87.5362°E, 43.9005°N	乌鲁木齐市周边
C103	13456	113.6850°E, 34.7306°N	郑州市周边
C106	13419	114.2952°E, 30.5824°N	合肥市中心城区

前 10 大聚类合计包含 259,109 条订单, 占有效订单总数的约 49.2%。其中, 上海市 (C31) 为最大聚类, 订单数达 58,156 条, 覆盖中心城区及周边区域; 广州、南京、天津等城市次之。该结果验证了医药物流需求的高度集中性, 头部效应显著——少数核心城市贡献了近一半的订单量。每个聚类将作为一个独立的子问题, 输入后续 ALNS 路径优化阶段。

4.5.2 路径优化: ALNS

采用自适应大邻域搜索 (Adaptive Large Neighborhood Search, ALNS) 算法求解配送方案。为评价每次迭代产生的新解, 首先定义综合得分函数, 然后通过破坏与修复操作迭代寻优。详细的算法伪代码及参数设置见附录 ??。

评价指标与权重 选取装载率 ($\bar{L}$)、平均里程 ($\bar{M}$)、温度适配率 (R_t)、非超载率 (R_o)、准时率 (R_{ot}) 五项指标。采用层次分析法 (AHP) 确定权重, 构造判断矩阵如表 8 所示 (1-9 标度法)。

表 8: 评价指标判断矩阵

	装载率	平均里程	温度适配率	非超载率	准时率
装载率	1	2	1/6	3	1/3
平均里程	1/2	1	1/9	2	1/5
温度适配率	6	9	1	9	2
非超载率	1/3	1/2	1/9	1	1/6
准时率	3	5	1/2	6	1

计算得最大特征值 $\lambda_{\max} = 5.067766$, 特征向量归一化后得到权重向量:

$$\mathbf{w} = (0.1044, 0.0609, 0.5166, 0.0415, 0.2766)$$

一致性检验: $CI = 0.016942$, $RI = 1.12$, $CR = 0.015126 < 0.1$, 通过。

综合得分函数

$$F = 0.1044\bar{L} + 0.0609(2 - \frac{\bar{M}}{500}) + 0.5166R_t + 0.0415R_o + 0.2766R_{ot}$$

(符号含义同数学模型，此处不再赘述)

算法总体框架

算法流程 ALNS 以最大化 F 为目标。流程：初始解 $\rightarrow$ 轮盘赌选破坏/修复算子 $\rightarrow$ 破坏/修复 $\rightarrow$ 模拟退火接受 $\rightarrow$ 每 25 次迭代更新权重（衰减 0.82）。破坏算子：随机、装载、温度；修复算子：贪心、随机。采用坐标缓存、惰性计算、浅拷贝等优化。接受准则采用模拟退火（初始温度 100，终止温度 0.05，降温系数自动计算），终止条件为最大迭代次数 500 或连续 80 次无改进。

算子设计 为在解空间中有效搜索，设计了以下三种破坏算子和两种修复算子，每次迭代以等概率选择其中之一执行。

破坏算子：

1. **随机破坏**：随机选择一辆车，从其配送路线中随机移除 k 个订单（ k 在 $[1, 3]$ 间均匀随机），被移除的订单存入暂存列表。
2. **装载破坏**：移除装载率偏离目标值（如 0.85）较大的车辆中的运单，通过优化低效装载来提升整体效能。
3. **温度破坏**：识别并移除温控属性不匹配（如常温车运送冷链单）的运单。

修复算子：

1. **贪心修复**：对被破坏算子移除的订单列表，逐个处理。对每个待插入订单，计算其插入到所有车辆路线的所有可能位置后目标函数 F 的增量，选择使 F 提升最多（或下降最少）的位置插入。
2. **随机修复**：随机选择一辆装载率未超过阈值的车辆，将订单插入其路线末尾，以保持搜索多样性。

每次迭代后，根据新解的质量给予算子奖励：成为全局最优奖励 25 分，改善当前解奖励 8 分，仅被退火接受奖励 1 分。每 25 次迭代，算子得分按衰减因子 0.82 与历史权重进行指数平滑，重新归一化选择概率，实现自适应调整。

性能优化策略 为应对大规模数据下 ALNS 的计算开销，本文实施了以下性能优化：

- **坐标预转换与缓存**：在数据加载阶段将所有经纬度坐标预转换为弧度，并采用缓存策略存储已计算的距离结果，避免运行时重复的三角函数运算。
- **惰性缓存机制**：在车辆类中引入里程、装载率等指标的惰性计算缓存，仅当订单序列发生改变时才重新计算，避免目标函数评价时的冗余计算。
- **对象浅拷贝**：利用运单对象的不可变性，在解的复制过程中共享运单实例，仅拷贝车辆容器，降低内存分配与垃圾回收压力。
- **向量化与局部变量提升**：将配置参数提取为循环内的局部变量，减少字典查找开销；对装载率等指标进行批量计算，提升运算效率。

4.6 求解结果展示与分析

4.6.1 收敛性与稳定性分析

ALNS 算法在迭代过程中的关键指标变化曲线如图 20 所示。从图中可以观察到以下特征：

综合得分与退火温度：综合得分随迭代次数单调上升，前期提升明显，后期趋于平缓，表明算法在迭代初期快速找到较优解，后期持续微调收敛。退火温度曲线快速下降并趋于稳定，说明算法接受劣解的概率逐步降低，从全局探索平滑过渡到局部优化阶段，有效避免了陷入局部最优。

运营效率指标：平均装载率从初始约 22% 提升至最终约 51%（提升超过一倍），平均里程从约 1074 km 下降至约 984 km，平均行驶时间从约 15.2 h 降至约 13.8 h。这些趋势表明算法通过拼单与路径优化显著提高了运力利用率，降低了距离成本与时间成本。

服务质量与合规指标：准时送达率从约 85.6% 提升至 87.6%，温度适配率稳定维持在 97.8% 以上并缓慢上升至 98.4%，超载率从 22.6% 微增至 23.9%（略有上升，后续可通过强化约束控制）。总体来看，优化方案在提升效率的同时兼顾了冷链安全与时效要求。



图 20: ALNS 全局优化指标曲线

4.6.2 优化前后指标对比

优化前后关键指标对比如表 9 所示。

表 9: 问题三优化前后关键指标对比

指标	优化前	优化后	变化
综合得分	0.79	0.84	+6.3%
平均装载率	21.73%	50.93%	+29.20%
超载率	22.60%	23.87%	+1.27%
平均里程 (km)	1074	984	-8.4%
平均行驶时间 (h)	15.15	13.84	-8.6%
温度适配率	97.76%	98.41%	+0.65%
准时送达率	85.57%	87.55%	+1.98%
总车辆数	75471	61010	-19.2%
总运单数	185124	185124	不变

4.6.3 运输效率与服务质量分析

平均装载率由 21.73% 提升至 50.93%，提升超过一倍；平均里程降低 8.4%，车辆总数减少 19.2%，显著降低了运营成本。温度适配率与准时送达率分别提升至 98.41% 和 87.55%，表明优化方案兼顾了冷链安全与时效。超载率略有上升 (22.60%→23.87%)，可在后续工作中通过强化约束予以控制。

4.7 问题三优化结论

ST-DBSCAN+ALNS 两阶段优化后，平均装载率升至 50.93% (+29.2%)，里程降 8.4%，车辆减 19.2%，综合得分 +6.3%，温度适配率 98.41%，准时率 87.55%，验证了优化有效性。

5 模型评价与改进

优点：成本核算精准；AHP 多目标加权综合优化；ST-DBSCAN+ALNS 高效处理 50 万级数据，装载率提升一倍以上，里程与车辆数显著下降。

缺点：超载率未完全消除 (约 24%)；直线距离近似低估实际里程；时效规则简化。

改进方向：加入超载硬约束；接入真实路网数据；采用 NSGA-II 等算法避免主观赋权。

参考文献

- [1] 托盘循环共用系统货物码盘及交接要求, 2025. 实施日期: 2026-01-01.
- [2] S. Alka et al. Cold chain logistics challenges on sustainability: A systematic review. *Sustainable Futures*, 10:101559, 2025.
- [3] 中华人民共和国商务部. 托盘循环共用系统货物码盘及交接要求, 2025. 行业标准 SB/T 11264-2025.
- [4] 广西壮族自治区交通运输厅. 道路货物运输价格与成本分析指南, 2022. 地方交通运输行业标准.

A ALNS 算法伪代码与参数设置

Algorithm 1 自适应大邻域搜索 (ALNS) 调度算法

Require: 初始解 s_0 (基于 ST-DBSCAN 聚类结果), 初始温度 $T_0 = 100.0$, 终止温度 $T_{\min} = 0.05$,

1: 最大迭代次数 $N = 100$, 早停阈值 $P = 80$, 权重集 W

Ensure: 最优调度方案 s^*

2: $s \leftarrow s_0, s^* \leftarrow s_0, T \leftarrow T_0$

3: 初始化算子权重 W_d, W_r 及得分记录器

4: **for** $iter = 1 \rightarrow N$ **do**

5: 根据轮盘赌机制选择破坏算子 $d \in \{\text{随机, 装载, 温度}\}$ 和修复算子 $r \in \{\text{贪心, 随机}\}$

6: 生成新解 $s' \leftarrow r(d(s))$ {包含车型映射与时空约束检查}

7: 计算得分差 $\Delta = F(s') - F(s)$ { F 为多目标评价函数}

8: **if** $\Delta > 0$ 或 $\exp(\Delta/T) > \text{random}(0, 1)$ **then**

9: $s \leftarrow s'$

10: **if** $F(s) > F(s^*)$ **then**

11: $s^* \leftarrow s$, 算子得分加成 σ_1

12: **else**

13: 算子得分加成 σ_2

14: **end if**

15: **else**

16: 算子得分加成 σ_3

17: **end if**

18: **if** $iter \pmod{25} == 0$ **then**

19: 利用衰减因子 $\rho = 0.82$ 更新算子权重 W_d, W_r

20: **end if**

21: $T \leftarrow T \times \text{cool_rate}$

22: **if** 连续 P 次迭代未改善 **then**

23: **break**

24: **end if**

25: **end for**

26: **return** s^*

B ST-DBSCAN 时空聚类算法伪代码

Algorithm 2 基于 KDTree 与图压缩优化的 ST-DBSCAN 聚类算法

Require: 原始运单数据集 D , 空间阈值 $\varepsilon_{\text{spatial}} = 15 \text{ km}$, 时间阈值 $\varepsilon_{\text{temporal}} = 0.8 \text{ 天}$, 最小样本数 $\text{MinPts} = 3$

Ensure: 带簇标签的结果集 D_{Result} , 聚类中心信息表 Centers

- 1: 清洗数据 D , 过滤无效坐标与时间, 将调度时间转换为相对天数 T_{days}
- 2: 生成去重特征键 $\text{Key} \leftarrow [\text{Round}(\text{纬度}), \text{Round}(\text{经度}), \text{Round}(T_{\text{days}})]$
- 3: 按 Key 去重得到唯一节点集 U_{Nodes} , 并保留映射 Idx_{Map}
- 4: 初始化空间搜索度数半径 $\text{Eps}_{\text{Deg}} \leftarrow \varepsilon_{\text{spatial}}/111.32$
- 5: 基于 U_{Nodes} 的空间坐标构建 cKDTree 索引树
- 6: 空间查询: 寻找距离 $\leq \text{Eps}_{\text{Deg}}$ 的近邻点对集合 $\text{Spatial}_{\text{Pairs}}$
- 7: 初始化有效时空边集 $\text{Valid}_{\text{Pairs}} \leftarrow \emptyset$
- 8: **for** 每对近邻点 $(u, v) \in \text{Spatial}_{\text{Pairs}}$ **do**
- 9: **if** $|T_{\text{days}}(u) - T_{\text{days}}(v)| \leq \varepsilon_{\text{temporal}}$ **then**
- 10: $\text{Valid}_{\text{Pairs}}.\text{append}((u, v))$
- 11: **end if**
- 12: **end for**
- 13: 统计节点度数得 Degree 数组
- 14: 标记核心点: $\text{Core}_{\text{Mask}} \leftarrow (\text{Degree} \geq \text{MinPts} - 1)$
- 15: 基于 $\text{Valid}_{\text{Pairs}}$ 构建 CSR 格式邻接表 Adj
- 16: 初始化标签 Labels 、访问状态 Visited 、 $\text{Cluster}_{\text{ID}} \leftarrow 0$
- 17: **for** 每个节点 $s \in U_{\text{Nodes}}$ **do**
- 18: **if** $\text{Core}_{\text{Mask}}[s]$ 且 $\neg \text{Visited}[s]$ **then**
- 19: 队列 $\text{Queue} \leftarrow \{s\}$, $\text{Visited}[s] \leftarrow \text{True}$
- 20: **while** $\text{Queue} \neq \emptyset$ **do**
- 21: 弹出 u , $\text{Labels}[u] \leftarrow \text{Cluster}_{\text{ID}}$
- 22: **for** 每个时空邻居 $v \in \text{Adj}[u]$ **do**
- 23: **if** $\text{Core}_{\text{Mask}}[v]$ 且 $\neg \text{Visited}[v]$ **then**
- 24: $\text{Visited}[v] \leftarrow \text{True}$, Queue 添加 v
- 25: **else if** $\neg \text{Core}_{\text{Mask}}[v]$ 且 $\text{Labels}[v] = -1$ **then**
- 26: $\text{Labels}[v] \leftarrow \text{Cluster}_{\text{ID}}$ {边界点}
- 27: **end if**
- 28: **end for**
- 29: **end while**
- 30: $\text{Cluster}_{\text{ID}} \leftarrow \text{Cluster}_{\text{ID}} + 1$
- 31: **end if**
- 32: **end for**
- 33: 通过 Idx_{Map} 还原标签到全量数据 D_{Result}
- 34: 聚合得各簇均值与样本数, 形成 Centers 表

C 核心代码片段

问题二 ST-DBSCAN 聚类核心代码 (Python)

以下代码实现了基于时空约束的 ST-DBSCAN 聚类算法, 其中包括 Haversine 距离计算、邻域查询及密度扩散逻辑。

Listing 1: ST-DBSCAN 核心实现

```

1 import numpy as np
2 from math import radians, cos, sin, asin, sqrt
3
4 def haversine_km(lon1, lat1, lon2, lat2):
5     """Haversine 公式计算球面距离 (km) """
6     lon1, lat1, lon2, lat2 = map(radians, [lon1, lat1, lon2, lat2])
7     dlon = lon2 - lon1
8     dlat = lat2 - lat1
9     a = sin(dlat/2)**2 + cos(lat1) * cos(lat2) * sin(dlon/2)**2
10    return 6371 * 2 * asin(sqrt(a))
11
12 class STDBSCAN:
13     """时空密度聚类 (ST-DBSCAN) """
14     def __init__(self, eps1=600, eps2=10, min_pts=3):
15         self.eps1 = eps1          # 空间阈值 (km)
16         self.eps2 = eps2          # 时间阈值 (天)
17         self.min_pts = min_pts
18
19     def fit(self, lons, lats, times):
20         n = len(lons)
21         labels = np.full(n, -1, dtype=int)
22         visited = np.zeros(n, dtype=bool)
23         cluster_id = 0
24
25         def neighbors(i):
26             """返回样本 i 的所有时空邻居索引"""
27             nb = []
28             for j in range(n):
29                 if (abs(times[i] - times[j]) ≤ self.eps2 and
30                     haversine_km(lons[i], lats[i], lons[j], ...
31                                 lats[j]) ≤ self.eps1):
32                 nb.append(j)
33             return nb
34
35         for i in range(n):
36             if visited[i]:
37                 continue
38             visited[i] = True
39             nb = neighbors(i)
40             if len(nb) < self.min_pts:
41                 labels[i] = -1
42                 # 标记为噪声
43             else:
44                 labels[i] = cluster_id
45                 queue = list(nb)
46                 while queue:
47                     q = queue.pop()
48                     if not visited[q]:
49                         visited[q] = True
50                         nb_q = neighbors(q)
51                         if len(nb_q) ≥ self.min_pts:
52                             queue.extend(nb_q)
53                     if labels[q] == -1:
54                         labels[q] = cluster_id
55                 cluster_id += 1
56
57         self.labels_ = labels
58         self.n_clusters_ = cluster_id

```

问题三 ST-DBSCAN 高效聚类核心代码 (Python)

本实现采用去重压缩、k-d Tree 空间索引、CSR 稀疏图存储及广度优先搜索，适用于 50 万级数据量。

Listing 2: ST-DBSCAN 高效实现

```

1 import numpy as np
2 import pandas as pd
3 from scipy.spatial import cKDTree
4
5 def st_dbscan_correct(df, eps_km, eps_days, min_samples):
6     """
7     时空密度聚类 (ST-DBSCAN) 高效版本
8     参数:
9         df: DataFrame, 需包含 '纬度', '经度', '时间天数' 列
10        eps_km: 空间半径 (km)
11        eps_days: 时间半径 (天)
12        min_samples: 最小邻域点数 (含自身)
13    """
14    # 1. 去重压缩: 对经纬度(6位)+时间(3位)生成唯一键
15    df['_key'] = (df['纬度'].round(6).astype(str) + "_" +
16                 df['经度'].round(6).astype(str) + "_" +
17                 df['时间天数'].round(3).astype(str))
18    unique_df = df.drop_duplicates('_key').reset_index(drop=True)
19    idx_map = {k: i for i, k in enumerate(unique_df['_key'])}
20    df['_uniq_idx'] = df['_key'].map(idx_map)
21
22    n = len(unique_df)
23    lat = unique_df['纬度'].values
24    lon = unique_df['经度'].values
25    tday = unique_df['时间天数'].values
26
27    # 2. 空间索引: k-d Tree 查询近邻
28    coords = np.stack([lat, lon], axis=1)
29    eps_deg = eps_km / 111.32 # 纬度1° 111.32 km
30    tree = cKDTree(coords)
31    pairs = np.array(list(tree.query_pairs(r=eps_deg)), dtype=np.int32)
32
33    if len(pairs) == 0:
34        unique_df['簇标签'] = -1
35        df['簇标签'] = -1
36        return df, unique_df
37
38    # 3. 时空双重约束: 筛选时间差 ≤ eps_days 的边
39    dt = np.abs(tday[pairs[:, 0]] - tday[pairs[:, 1]])
40    valid_mask = dt ≤ eps_days
41    vpairs = pairs[valid_mask]
42
43    # 4. 计算节点度数, 标记核心点
44    deg = np.zeros(n, dtype=np.int32)
45    np.add.at(deg, vpairs[:, 0], 1)
46    np.add.at(deg, vpairs[:, 1], 1)
47    core_mask = deg ≥ (min_samples - 1)

```

```

48
49 # 5. 构建 CSR 格式邻接表 (压缩稀疏行, 节省内存)
50 row = np.concatenate([vpairs[:, 0], vpairs[:, 1]])
51 col = np.concatenate([vpairs[:, 1], vpairs[:, 0]])
52 order = np.argsort(row, kind='mergesort')
53 row = row[order]
54 col = col[order]
55 ptr = np.zeros(n + 1, dtype=np.int32)
56 np.add.at(ptr[1:], row, 1)
57 np.cumsum(ptr, out=ptr)
58
59 # 6. BFS 扩展聚类
60 labels = np.full(n, -1, dtype=np.int32)
61 visited = np.zeros(n, dtype=bool)
62 cluster_id = 0
63 for s in range(n):
64     if not core_mask[s] or visited[s]:
65         continue
66     queue = [s]
67     visited[s] = True
68     while queue:
69         u = queue.pop(0)
70         labels[u] = cluster_id
71         for v in col[ptr[u]:ptr[u+1]]:
72             if core_mask[v] and not visited[v]:
73                 visited[v] = True
74                 queue.append(v)
75             elif not core_mask[v] and labels[v] == -1:
76                 labels[v] = cluster_id # 边界点归入簇
77     cluster_id += 1
78
79 # 7. 整理标签并还原到全量数据
80 valid_labels = sorted(set(labels[labels != -1]))
81 label_map = {-1: -1}
82 for new_id, old_id in enumerate(valid_labels):
83     label_map[old_id] = new_id
84 unique_df['簇标签'] = np.vectorize(label_map.get)(labels)
85 df['簇标签'] = unique_df['簇标签'].values[df['_uniq_idx']]
86 return df, unique_df

```

模拟退火核心代码 (问题二)

Listing 3: 模拟退火主循环与邻域操作

```

1 import random
2 import copy
3 import math
4
5 T0 = 3000 # 初始温度
6 r = 0.997 # 降温系数
7 Ts = 0.01 # 终止温度
8
9 def full_copy(df):
10     """深拷贝订单DataFrame"""
11     new_df = df.copy()
12     for i in range(len(new_df)):

```

```

13         new_df.loc[i, "订单"] = copy.deepcopy(new_df.loc[i, "订单"])
14     return new_df
15
16 def SA(month_index):
17     """模拟退火求解该月最优调度"""
18     best = full_copy(initial_solution)
19     best_cost = calc_cost(best)
20     current = full_copy(initial_solution)
21     T = T0
22     iterate_costs = []
23
24     while T > Ts:
25         cost_old = calc_cost(current)
26
27         # 邻域操作1 (概率0.7) : 同一车辆内交换两个订单 + 车型调整
28         if random.random() < 0.7:
29             new_sol = full_copy(current)
30             vehicle = random.randint(0, len(new_sol)-1)
31             route = new_sol.loc[vehicle, "订单"]
32             if len(route) >= 2:
33                 a, b = random.sample(range(len(route)), 2)
34                 route[a], route[b] = route[b], route[a]
35                 new_sol.loc[vehicle, "订单"] = route
36                 # 按熵权概率更换车型
37                 rnd = random.random()
38                 if rnd <= 0.39:
39                     new_sol.loc[vehicle, "车型"] = "4.2M"
40                 elif rnd <= 0.77:
41                     new_sol.loc[vehicle, "车型"] = "7.6M"
42                 else:
43                     new_sol.loc[vehicle, "车型"] = "9.6M"
44             else:
45                 # 邻域操作2: 将一个订单从一辆车迁移到另一辆车
46                 new_sol = full_copy(current)
47                 src = random.randint(0, len(new_sol)-1)
48                 route_src = new_sol.loc[src, "订单"]
49                 if route_src:
50                     order = random.choice(route_src)
51                     route_src.remove(order)
52                     new_sol.loc[src, "订单"] = route_src
53                     dst = random.randint(0, len(new_sol)-1)
54                     route_dst = new_sol.loc[dst, "订单"]
55                     pos = random.randint(0, len(route_dst))
56                     route_dst.insert(pos, order)
57                     new_sol.loc[dst, "订单"] = route_dst
58
59                 cost_new = calc_cost(new_sol)
60                 Δ = cost_new - cost_old
61
62                 if Δ < 0 or random.random() < math.exp(-Δ / T):
63                     current = new_sol
64                     if cost_new < best_cost:
65                         best = new_sol
66                         best_cost = cost_new
67
68                 T *= r
69                 iterate_costs.append(best_cost)
70

```

```
71     # 绘制收敛曲线（省略绘图细节）  
72     return best
```